CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem



Bowen Liu

Bowen Liu 刘博文

Periodic N-body solutions, elliptic relative equilibria and linear stability.

2013–2018 **Nankai University** · Chern Institute · Math PhD

2018–2020 **University of Augsburg** · Postdoc

2020–2022 **Shanghai Jiao Tong** · Wu Wen-Tsun Asst. Prof.

2022–2024 **Huawei** · Principal Engineer

2024–now **US e-commerce** · Applied Researcher

CELESTIAL MECHANICS RESEARCH

Periodic N-body: existence & stability

Restricted four-body elliptic equilibria

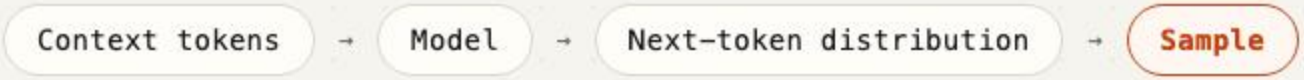
Linear instability of elliptic rhombi

Homographic Lennard–Jones solutions

> Symplectic indices · Variational methods

The model only predicts the next token

Memory, computation, action and goals live outside that action.



01 · NO SAVED STATE

The client resends the history

02 · COMPUTATION IS RECALL

Like a times table: memory, not an algorithm

03 · NO DIRECT ACTION

Tools execute outside

04 · NO PERSISTENT GOAL

Plans are generated text

Four limits → three system capabilities: **state, computation, reliability**; probabilistic output needs verification, stopping, rollback.

> The model gives only the next token; **everything else comes from the system**.

Put the model back in the system



01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> Memory owns state, tool calls compute, harness owns bounds: together they form an **agent system**.

Calculation, or recall?

The model predicts the next token; it does not execute multiplication.

13 × 15

$$
\begin{aligned}
&13 \times 15 \\
&= 13 \times 5 + 13 \times 10 \\
&= 65 + 130 \\
&= 195
\end{aligned}
$$

> 3 × 5 = 15 is recall; decomposition and addition are still token prediction.

01 · TIMES TABLE

3 × 5 = 15 is memorized, not computed.

02 · DECOMPOSITION

13 × 15 can be broken into known pieces; decomposing is not arithmetic.

03 · SAMPLING

When data is sparse, the distribution is flat; sampling can pick a wrong answer.

> Deterministic problems need deterministic results; **exact computation belongs to a program**.

Harness: bound the tools, recover the run

Tools execute in an environment; probabilistic output will eventually fail on long runs.

A controlled loop

Propose → Validate → Execute → Observe → Verify → Decide

TWO PROBLEMS

Tools must not cross bounds: no deleting files, no public uploads.

Long runs must roll back: checkpoints, snapshots, hard stops.

It does not add intelligence; it makes errors **visible, stoppable, reproducible**.

01 TOOL REGISTRATION & VALIDATION
Check schemas and types; block illegal proposals immediately

02 SANDBOX & PERMISSIONS
Separate read and write; restrict permissions, network and directory scope before execution

03 TIMEOUTS & RETRIES
Prevent infinite loops; enforce timeout limits and retry caps

04 RESULT VERIFICATION
Check correctness via independent assertions; never trust self-claims

05 ROLLBACK & STOPPING
Checkpoints, snapshots, idempotency; force a stop on exception

06 LOGS & REPLAY
Record every call and response; ensure full trajectory auditability

> The model provides probability; the harness provides **bounds and a way back**.

Theorem-cracking system: LLM as CPU

Not just a mathematics problem; a machine for throughput and stability.

WORKLOAD	**Theorem · proof graph**	what to solve
CPU	**LLM**	propose next
SCHEDULER	**Loop + Harness**	concurrency · retries · checkpoints · stop
VERIFIER	**Lean / tools**	correctness · compute
STATE	**Context + Memory**	graph · logs · budget

01 · NOT ONLY MATHEMATICS

Decomposition, scheduling, state, verification

02 · THROUGHPUT

Concurrency, caching, reclamation, cost

03 · STABILITY

Long runs, recovery, auditability

04 · SCALABLE

Swap problems, not the system

> The model is the CPU; **theorem cracking needs the whole machine**.

Failure probability: long chains amplify 1%

The proof graph is read and written constantly; errors compound along dependencies.

END-TO-END SUCCESS · ILLUSTRATIVE

Per step	100	1,000	10,000
99%	37%	0.0043%	≈0
99.9%	90%	37%	0.0045%
99% + 3 retries	≈100%	99.9%	99%

Assumes independent failures; real errors correlate, and retries are not free.

DETECT ERRORS

Lean, tests, assertions first

RETRY SAFELY

Backoff, caps, no duplicate effects

ROLL BACK STATE

Checkpoints, idempotency, recovery

> The system does not prevent every error; **it makes errors non-fatal**.

Systems engineering: six interfaces

Each layer can work alone; the whole machine still may not be reliable.

01 · STATE CONSISTENCY
Concurrent graph edits
Versions, locks, conflict merge

02 · SCHEDULING & BUDGET
Who runs, how long
Concurrency, preemption, cost

03 · EXECUTION FAILURE
Execution errors / code bugs
Idempotency, backoff, caps

04 · VERIFICATION INTERFACE
Natural language ↔ Lean
Semantic gaps and assumptions

05 · OBSERVABILITY
Which layer failed?
Execution · program · tool · search path

06 · HUMAN BOUNDARY
When to stop, who owns
Automate execution, gate judgment

Low-hanging fruit: hard to prove, easy to disprove; one counterexample settles it · **Real systems problem**: reproducible, scalable, maintainable

Low-hanging fruit proves direction; **the hard part is the whole machine**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → `theorem`

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.

Thank you